

AS Statement

AS clause helps us to create aliases (rename columns to whatever we want). It also improves the readability of a query result.

Example syntaxes:

```
SELECT column AS new_name FROM table;
```

```
SELECT SUM(column) AS new_name FROM table;
```

Example usecase:

Query Editor

Query History

1

SELECT amount AS rental_price

2

FROM payment;

Data Output

Explain

Messages

Notifications

	rental_price numeric (5,2)	
1	7.99	
2	1.99	

Query Editor

Query History

1

SELECT SUM(amount) AS net_revenue

2

FROM payment;

Data Output

Explain

Messages

Notifications

	net_revenue numeric	
1	61312.04	

The **AS** operator gets executed at the very end of a query, meaning that we can not use the alias inside a **WHERE** operator.

JOINs Operation

JOIN clause allows us to combine information from multiple tables. There are many different **JOIN** types because it helps to decide how to deal with information only present in one of the joined tables. Types of joins are:

1. Inner Join
2. Outer Joins
 - a. LEFT OUTER JOIN
 - b. RIGHT OUTER JOIN

INNER JOIN

Let's imagine a simple example.
Our company is holding a conference for people in the movie rental industry.
We'll have people register online beforehand and then login the day of the conference.

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

For the sake of simplicity, we will assume the names are unique.

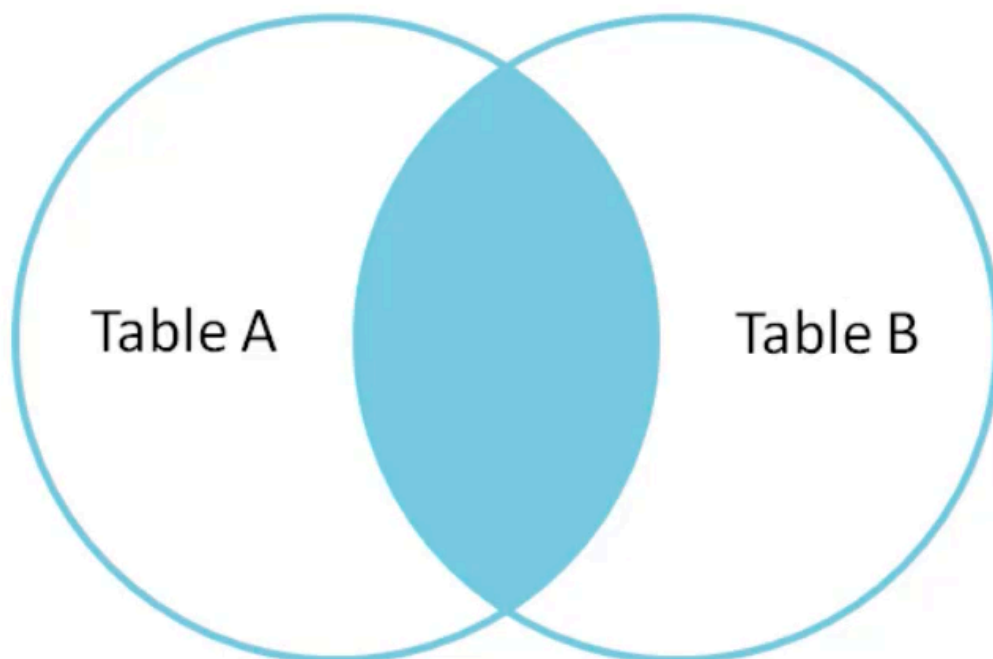
An INNER JOIN will result with the set of records that match in both tables.

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

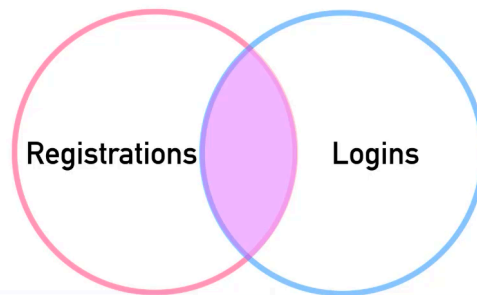
Example syntax:

```
SELECT * FROM TableA
INNER JOIN TableB
ON TableA.col_match = TableB.col_match
```



```
SELECT * FROM Registrations
INNER JOIN Logins
ON Registrations.name = Logins.name;
```

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David



LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob

```
SELECT reg_id, Logins.name, log_id
FROM Registrations
INNER JOIN Logins
ON Registrations.name = Logins.name;
```

RESULTS		
reg_id	name	log_id
1	Andrew	2
2	Bob	4

You need to specify the name column alongside the table name because the column exists in both tables.

Important Notes:

- Remember that table order won't matter in an **INNER JOIN**.
- Also if you see just **JOIN** without the **INNER**, PostgreSQL will treat it as an **INNER JOIN**.

OUTER JOINS

There are few different types of OUTER JOINS, which allows us to deal with values only present in one of the tables being joined. They are much more complex JOINS than the simpler INNER JOINS.

Lets review our 2 example tables from the previous lectures.

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

In the above tables, we only have names that only appear in one table.

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

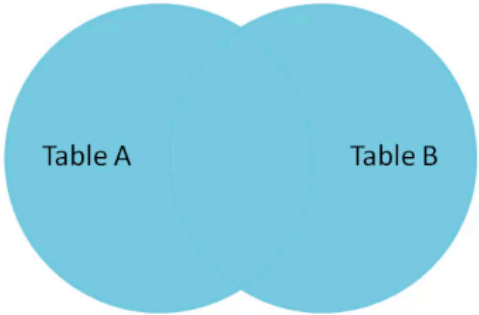
LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

There are mainly 3 types of outer joins:

1. FULL OUTER JOIN

OUTER JOIN or an FULL OUTER JOIN, just grabs everything.

```
SELECT * FROM TableB
FULL OUTER JOIN TableA
ON TableA.col_match = TableB.col_match
```

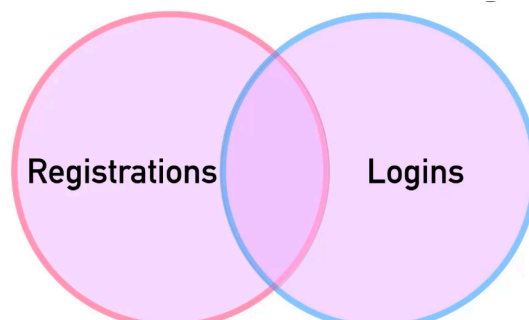


AN DATA

Example:

```
SELECT * FROM registrations
FULL OUTER JOIN logins
ON registrations.name = logins.name;
```

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David



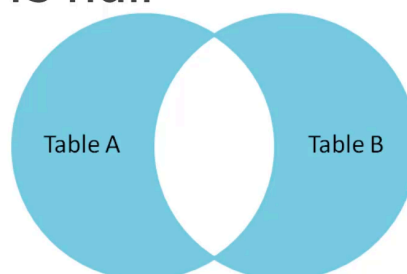
LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

REGISTRATIONS		RESULTS				LOGINS	
reg_id	name	reg_id	name	log_id	name	log_id	name
1	Andrew	1	Andrew	2	Andrew	1	Xavier
2	Bob	2	Bob	4	Bob	2	Andrew
3	Charlie	3	Charlie	null	null	3	Yolanda
4	David	4	David	null	null	4	Bob
		null	null	1	Xavier		
		null	null	3	Yolanda		

→ **FULL OUTER JOIN with WHERE** : Gets us rows unique to either table, in other words rows that are not found in both tables. Essentially this is the exact opposite of an **INNER JOIN**.



- **SELECT * FROM TableA**
FULL OUTER JOIN TableB
ON TableA.col_match = TableB.col_match
WHERE TableA.id IS null OR
TableB.id IS null



In our previous result, we had

- **SELECT * FROM Registrations FULL OUTER JOIN Logins
ON Registrations.name = Logins.name**

REGISTRATIONS		RESULTS				LOGINS	
reg_id	name	reg_id	name	log_id	name	log_id	name
1	Andrew	1	Andrew	2	Andrew		
2	Bob	2	Bob	4	Bob		
3	Charlie	3	Charlie	null	null	1	Xavier
4	David	4	David	null	null	2	Andrew
		null	null	1	Xavier	3	Yolanda
		null	null	3	Yolanda	4	Bob

We have null values for our registration ID and no values for log ID. Technically, you can also check null values on name columns as well, but it's better to use only one of the above columns to filter the result.

```
SELECT * FROM registrations  
FULL OUTER JOIN logins  
ON registrations.name = logins.name  
WHERE registrations.reg_id IS null OR logins.log_id IS null;
```

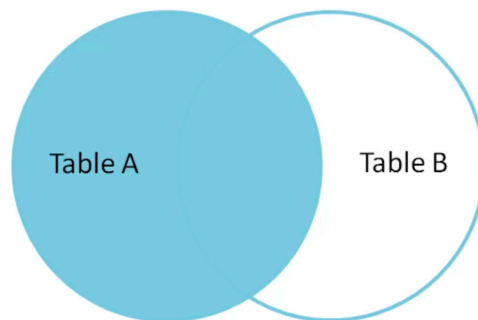
REGISTRATIONS		RESULTS				LOGINS	
reg_id	name	reg_id	name	log_id	name	log_id	name
1	Andrew	3	Charlie	null	null	1	Xavier
2	Bob	4	David	null	null	2	Andrew
3	Charlie	null	null	1	Xavier	3	Yolanda
4	David	null	null	3	Yolanda	4	Bob

This essentially removes the rows which had similar names on both tables (Andrew and Bob) and shows us the results that are unique to only registrations and logins. Essentially the exact opposite of **INNER JOIN**.

2. LEFT OUTER JOIN

A **LEFT OUTER JOIN** results in the set of records that are in the left table, if there is no match with the right table, the results are null. You can also add **WHERE** statements to further modify a **LEFT OUTER JOIN**.

- **SELECT * FROM TableA**
LEFT OUTER JOIN TableB
ON TableA.col_match = TableB.col_match



PIERIAN DATA

Odemy

- *Note that the results for this query are not symmetrical, so the order of the table matters.*
- *Also you can use LEFT JOIN instead of LEFT OUTER JOIN in SQL syntax.*

Lets explore a **LEFT OUTER JOIN** with our two example tables.

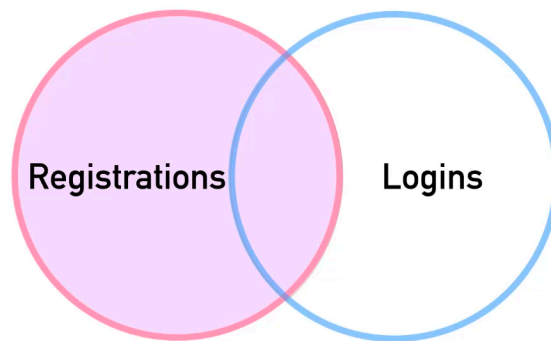
REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

Query:

```
SELECT * FROM registrations
LEFT OUTER JOIN logins
ON registrations.name = logins.name;
```

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David



LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

PIERIAN DATA

Table

- **SELECT * FROM Registrations
LEFT OUTER JOIN Logins
ON Registrations.name = Logins.name**

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

PIERIAN DATA

Table

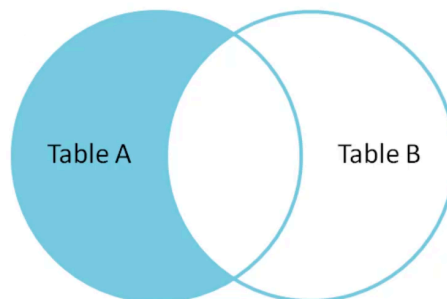
Everything from the left table (registrations) ends up in the result. Which is why it is critical to remember that order matters.

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

- What if we only wanted entries unique to Table A? Those rows found in Table A and not found in Table B.



→ **LEFT OUTER JOIN WITH WHERE** : Gets rows unique to left table.

Recall that in our previous query, the log_id columns have some null values.

- SELECT * FROM Registrations
LEFT OUTER JOIN Logins
ON Registrations.name = Logins.name**

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

The login_id only exists for those whose names are in the right table (logins) and also in the left table (registrations). So the rows with these nulls (Charlie and David) are exclusive to the left hand table (registration). Therefore, they are unique to the left table (registrations).

**SELECT * FROM Registrations
LEFT OUTER JOIN Logins
ON Registrations.name = Logins.name
WHERE Logins.log_id IS null**

REGISTRATIONS	
reg_id	name
1	Andrew
2	Bob
3	Charlie
4	David

RESULTS			
reg_id	name	log_id	name
3	Charlie	null	null
4	David	null	null

LOGINS	
log_id	name
1	Xavier
2	Andrew
3	Yolanda
4	Bob

PIERIAN DATA

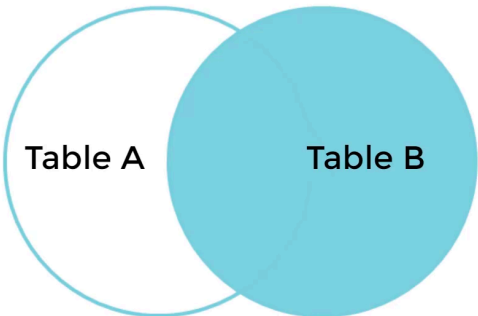
Odemy

3. RIGHT OUTER JOIN

A **RIGHT JOIN** is essentially the same as a **LEFT JOIN**, except the tables are switched. Therefore, this would be the same as switching the table order in a **LEFT OUTER JOIN**.

Query example:

- SELECT * FROM TableA
RIGHT OUTER JOIN TableB
ON TableA.col_match = TableB.col_match**

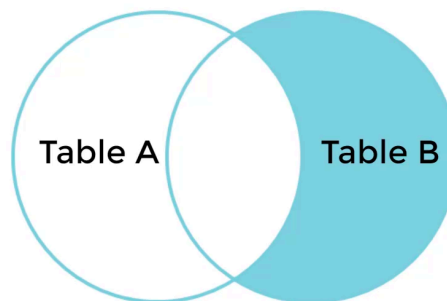


PIERIAN DATA

The query returns back rows that can be found either exclusively in table B or in both table A and B. It doesn't return rows that are exclusively only found in table A. You can use **RIGHT JOIN** or **RIGHT OUTER JOIN** interchangeably.

→ **RIGHT OUTER JOIN WITH WHERE** : Gets rows that are exclusive to table B.

- **SELECT * FROM TableA**
RIGHT OUTER JOIN TableB
ON TableA.col_match = TableB.col_match
WHERE TableA.id IS null



PIERIAN  DATA



- It is up to you and how you have the tables organized “in your mind” when it comes to choosing a LEFT vs RIGHT join, since depending on the table order you specify in the JOIN, you can perform duplicate JOINS with either method.

UNION

The **UNION** operator is used to combine the result-set of two or more **SELECT** statements. It basically serves to directly concatenate two results together, essentially pasting them together.

Syntax example:

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

Note that while performing a **UNION**, they should be logical and they should match up in a way so that you can stack the results right on top of another.

Lets, go through an simple example:

Let's explore a **UNION** with two example tables.

Sales2021_Q1	
name	amount
David	100
Claire	50

Sales2021_Q2	
name	amount
David	200
Claire	100


```
SELECT * FROM Sales2021_Q1
UNION
SELECT * FROM Sales2021_Q2;
```

name	amount
David	100
Claire	50
David	200
Claire	100

You can also use ORDER BY at the end to order the result based on the name column.

- ```
SELECT * FROM Sales2021_Q1
UNION
SELECT * FROM Sales2021_Q2
ORDER BY name;
```

| name   | amount |
|--------|--------|
| David  | 100    |
| David  | 200    |
| Claire | 50     |
| Claire | 100    |